Rockchip SecurityBoot和AVB操作指南

文件标识：RK-KF-YF-E78

发布版本：V1.0.0

日期：2026-06-08

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档提供SecurityBoot和AVB的配置方法。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	吴良清	2026-06-08	基于各芯片平台文档整合

目录

Rockchip SecurityBoot和AVB操作指南

1. 支持芯片
2. 代码环境
3. 安全启动SecurityBoot操作步骤
   - 3.1 编译环境确认
   - 3.2 进入u-boot目录
   - 3.3 代码修改
   - 3.4 生成keys
   - 3.5 编译签名
   - 3.6 FIT独立签名
     - 3.6.1 签名流程
     - 3.6.2 签名脚本
   - 3.7 编译完整固件
   - 3.8 固件烧写
   - 3.9 判断是否熔断成功
     - 3.9.1 通过开机串口log判断
     - 3.9.2 烧写未签名的loader和uboot来判断
4. Android Verified Boot (AVB) 操作步骤
   - 4.1 关于AVB Key管理机制
   - 4.2 编译avbtool工具
   - 4.3 生成permanent_attributes.bin
     - 4.3.1 方式一：Android14及之前的版本（ATX机制）
     - 4.3.2 方式二：Android15及之后版本（Cert机制）
   - 4.4 代码修改
     - 4.4.1 device/rockchip BoardConfig.mk修改
     - 4.4.2 u-boot defconfig修改
   - 4.5 AVB key烧写
     - 4.5.1 方式一：AVB key烧写工具烧写（推荐量产使用）
       - 4.5.1.1 Android14及之前版本（ATX机制）
       - 4.5.1.2 Android15及之后（Cert机制）
     - 4.5.2 方式二：AVB key集成到uboot代码内，跟固件一起烧写到机器内
       - 4.5.2.1 该方式SDK默认没有支持，需要在uboot下打上补丁
       - 4.5.2.2 抽取公钥
       - 4.5.2.3 替换公钥
   - 4.6 固件编译
   - 4.7 固件烧写
   - 4.8 启动验证
   - 4.9 AVB严格校验的解锁步骤

1. 支持芯片

芯片平台	SecurityBoot	AVB	最低Android版本要求
RK3588	支持	支持	Android12 RKR4
RK3576	支持	支持	Android14
RK3572	支持	支持	Android16
RK3568	支持	支持	Android11 RKR7
RK3566	支持	支持	Android11 RKR7
RK3562	支持	支持	Android13
RK3326	支持	支持	Android11 RKR7
PX30	支持	支持	Android11 RKR7

注意：

- RK3399系列仅适用于AVB部分，SecurityBoot部分请参考文档《Rockchip_RK3399_User_Guide_SecurityBoot_And_AVB_CN.md》
- 本文档以RK3576/RK3588/RK356x为例，其他芯片平台的操作步骤类似，只需将对应平台名称替换即可

2. 代码环境

- Android11 RKR7及之后版本支持SecurityBoot和AVB功能
- 各芯片平台最低Android版本要求请参见上表

3. 安全启动SecurityBoot操作步骤

3.1 编译环境确认

确认编译服务器的fdtput版本是否是1.4.5版本

```
$ fdtput --version
Version: DTC 1.4.5
```

如果fdtput版本小于1.4.5，请执行如下命令升级

```
sudo apt-get install device-tree-compiler
```

3.2 进入u-boot目录

```
cd u-boot
```

以下操作步骤都在u-boot目录下执行

3.3 代码修改

进入u-boot目录，打开对应平台的defconfig文件，选择如下配置：

芯片平台	**defconfig**文件
RK3588	configs/rk3588_defconfig
RK3576	configs/rk3576_defconfig
RK3572	configs/rk3572_defconfig
RK3568/RK3566	configs/rk3568_defconfig
RK3562	configs/rk3562_defconfig
RK3326	configs/rk3326_defconfig
PX30	configs/px30_defconfig

```
# Edit the defconfig file for the corresponding platform
$ vim configs/rk3576_defconfig
// 必选。
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_AVB_VBMETA_PUBLIC_KEY_VALIDATE=y

// 可选：如需要支持RSA4096 keys需要开启
CONFIG_FIT_ENABLE_RSA4096_SUPPORT=y

// 可选。
CONFIG_FIT_ROLLBACK_PROTECT=y        // boot.img防回滚
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y    // uboot.img防回滚
```

说明： AVB keys默认是RSA2048，如果要支持RSA4096，需要打上补丁，详见：https://redmine.rock-chips.com/issues/632050

3.4 生成keys

支持生成rsa2048和rsa4096 keys，在u-boot目录下生成对应key的方式如下，可以根据需求选择一种方式：

注意： 以下步骤生成的keys为测试密钥（testkey），仅用于开发验证。量产产品必须使用独立生成的发布密钥，禁止使用默认的testkey。

生成rsa2048 keys：

```
mkdir -p keys
../rkbin/tools/rk_sign_tool kk --bits 2048 --out .
cp privateKey.pem keys/dev.key && cp publicKey.pem keys/dev.pubkey
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

生成rsa4096 keys：

```
mkdir -p keys
../rkbin/tools/rk_sign_tool kk --bits 4096 --out .
cp privateKey.pem keys/dev.key && cp publicKey.pem keys/dev.pubkey
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

注意：该步骤执行一次即可，然后妥善保存这些keys。`--burn-key-hash`会导致芯片熔断，此操作为不可逆操作，请务必确认后再执行。

3.5 编译签名

以下以RK3576为例，其他平台将rk3576替换为对应平台名称即可：

```
./make.sh rk3576 --spl-new --rollback-index-uboot 1 --burn-key-hash
```

说明：

- --spl-new：必选。重新打包签名后的spl
- --rollback-index-uboot <版本号>：可选。设置版本号，当步骤“代码修改”中的config配置了防回滚时，需要增加这个编译选项，否则不需要
- --burn-key-hash：可选。加这个编译选项就会在烧写固件后开机的时候进行芯片熔断。

如果编译出现：

```
Can't load XXXXXX//.rnd into RNG
```

执行：

```
touch ~/.rnd
```

3.6 FIT独立签名

有些客户不使用上面编译签名方式，需要使用服务器独立签名方式，Android平台只需要签名loader和uboot.img。

3.6.1 签名流程

编译固件的人员：

1. 参考前面的安全启动章节，打开相关配置项。
2. 编译：

```
./make.sh rk3576 --spl-new --no-sign    # 必须追加 --spl-new --no-sign
```

编译后会生成：

```
uboot.img
rk3576_spl_loader_v1.10.108.bin
fit/fit_signcfg/
```

3. 把生成的三个文件发给负责签名的人员或服务器。

   注意： 生成的loader、uboot跟fit_signcfg有严格的匹配关系，不能跟其他时刻编译的同名固件混在一起参与签名，否则签名脚本会报错。

签名的人员或服务器：

1. 创建工作目录（名字可任意）：

```
$ mkdir workspace
$ cd workspace
$ mkdir keys src output
```

2. 下载rkbin仓库，把rkbin里的tools目录拷贝或软链接到workspace目录下：

```
$ cp -r ~/rkbin/tools/ ./
```

3. 参考前面的安全启动章节把RSA密钥对放到keys目录下，把待签名的固件和fit_signcfg目录放到src目录下。

   其中loader、uboot和fit_signcfg是必须的。例如：

```
$ tree
.
├── keys
│   ├── dev.crt
│   ├── dev.key
│   └── dev.pubkey
├── output
├── src
│   ├── fit_signcfg
│   │   ├── MINIALL.ini
│   │   ├── sign.readonly_config
│   │   └── test_images.tar.gz
│   ├── rk3576_spl_loader_v1.10.108.bin
│   └── uboot.img
└── tools
    ├── boot_merger
    ├── burner_image_kits
    │   ├── make.sh
    │   └── make_slc_nand.sh
    ......
```

4. 执行签名：

```
$ ./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/
```

签名过程不会对src目录的源文件做任何修改。签名成功后会在output目录下生成所有签名固件：

```
$ tree output/
output/
├── rk3576_idblock_v1.10.108.img
├── rk3576_spl_loader_v1.10.108.bin      // 等同全编译MiniLoaderAll.bin
└── uboot.img
```

3.6.2 签名脚本

独立签名的相关工具都已提交在rkbin仓库，供开发者调用的脚本是：./tools/fit-sign.sh。

```
$ ./tools/fit-sign.sh -h

Usage:
    ./tools/fit-sign.sh [args]

Args:
    --key-dir                 <dir>                          | Mandatory
    --src-dir                 <dir>                          | Mandatory
    --out-dir                 <dir>                          | Mandatory
    --burn-key-hash                                          | Optional
    --rollback-index          <image1 n1> <image2 n2> ...    | Optional
    --version                 <image1 n1> <image2 n2> ...    | Optional
```

- --key-dir：必选。指向RSA密钥目录。
- --src-dir：必选。指向待签名固件目录，该目录必须包含loader、uboot、fit_signcfg目录。
- --out-dir：必选。指向签名后的输出目录。
- --burn-key-hash：可选。不需要跟额外参数，用于使能SPL烧写RSA公钥hash到otp。
- --rollback-index：可选。给对应固件指定防回滚版本号，十进制表示。
- --version：可选。给对应固件指定常规的文件版本号，十进制表示。

例如：

```
./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/ --version
uboot.img 1 boot.img 3 --rollback-index uboot.img 3 boot.img 5
```

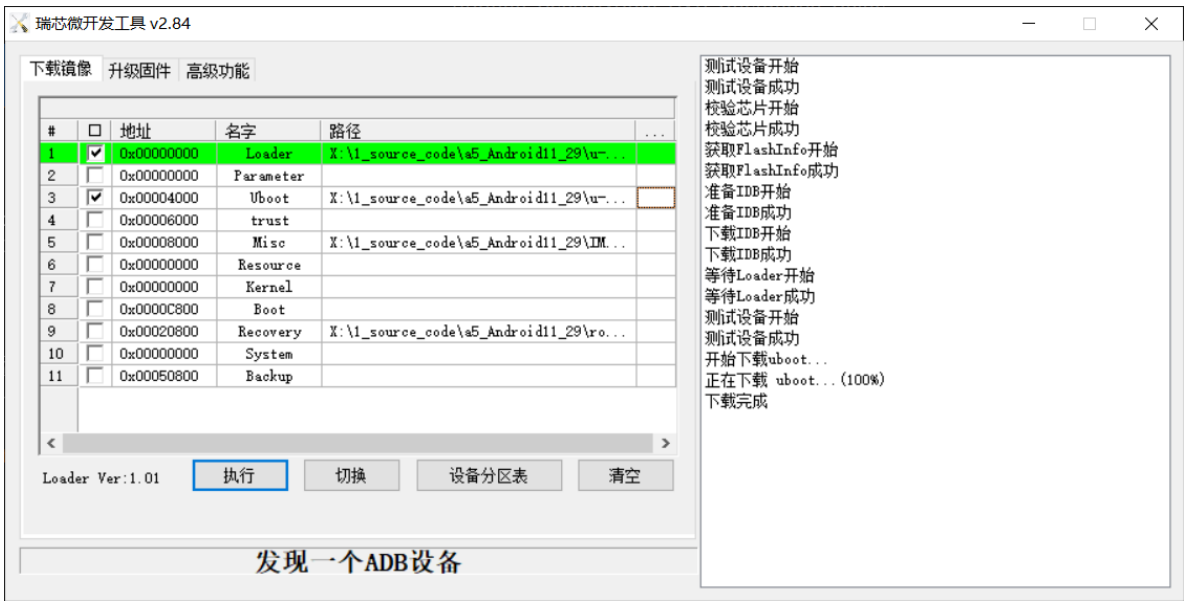
3.7 编译完整固件

按正常编译固件的方式编译其他固件（uboot和loader上面已经编译完，不需要重新编译），如用

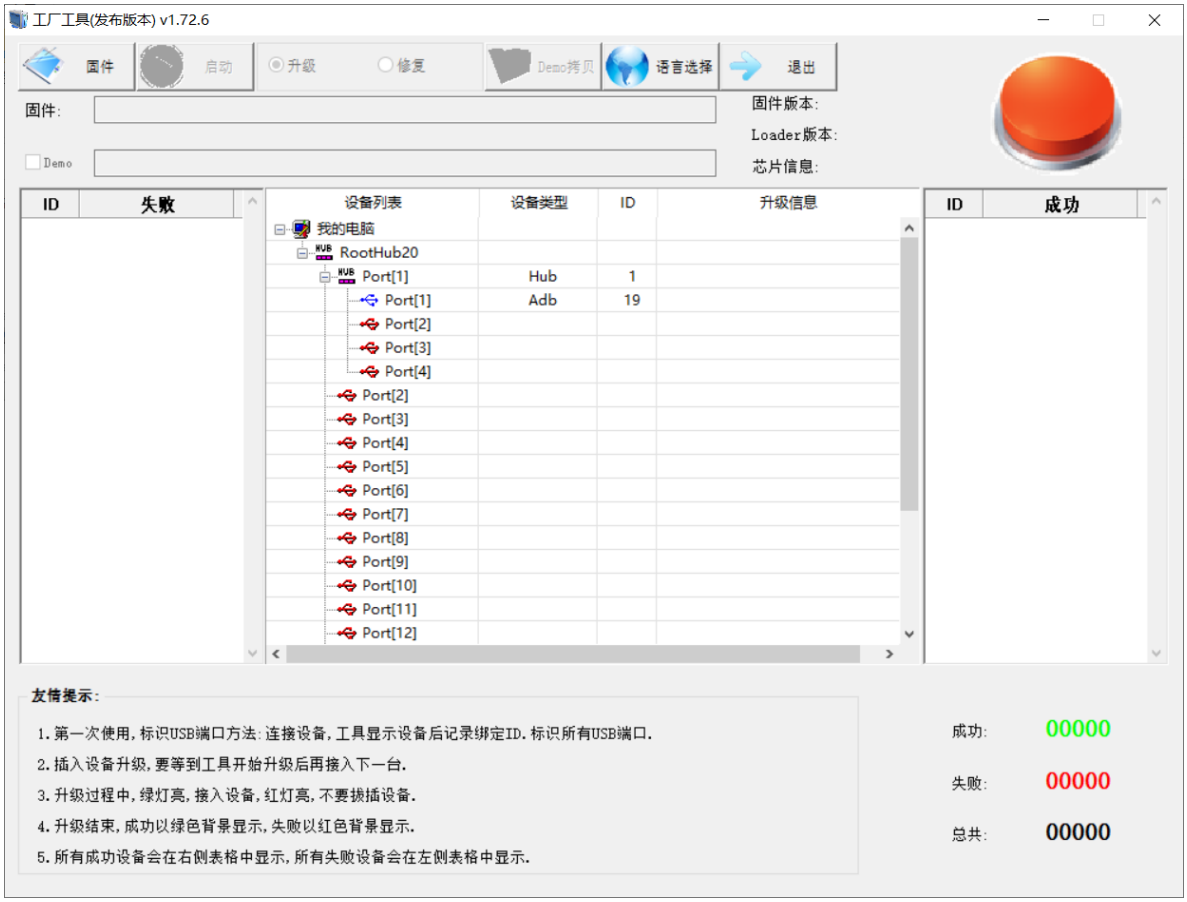
```
source build/envsetup.sh
lunch rk3576_u-userdebug
./build.sh -ACKup
```

3.8 固件烧写

开发验证阶段可以使用AndroidTool单独烧写uboot和loader即可



量产阶段使用量产工具烧写步骤“编译完整固件”中编译生成的update.img



3.9 判断是否熔断成功

3.9.1 通过开机串口log判断

```
### Verified-boot: 0    //固件签名但是芯片没有熔断（1表示有熔断）,没有进行hash有效性验证
sha256,rsa2048:dev+
rollback index: 1 >= 0(min), OK    //回滚版本
//下面这些是uboot完整性的校验
## Checking atf-1 0x00040000 ... sha256+ OK
## Checking uboot 0x00a00000 ... sha256+ OK
## Checking fdt 0x00b2a018 ... sha256+ OK
## Checking atf-2 0xfdcc9000 ... sha256+ OK
## Checking atf-3 0xfdcd0000 ... sha256+ OK
## Checking optee 0x00200000 ... sha256+ OK
```

3.9.2 烧写未签名的loader和uboot来判断

烧写未签名的loader或者uboot，开机后系统校验会失败，导致无法开机。

4. Android Verified Boot (AVB) 操作步骤

4.1 关于AVB Key管理机制

Rockchip平台根据Android版本不同，使用不同的AVB Key管理机制：

Android版本	**Key管理机制**	**Key文件名前缀**	**描述**
Android15及之后	Cert	testkey_cert*/*cert*	证书认证机制
Android14及之前	ATX	testkey_atx*/*atx*	传统ATX认证机制

注意： ATX和Cert机制的密钥文件、配置项名称有所不同，请根据实际使用的Android版本选择对应的章节操作。

4.2 编译avbtool工具

```
mmma external/avb/ -j16
```

编译完成后生成：

```
out/host/linux-x86/bin/avbtool
```

4.3 生成permanent_attributes.bin

根据Android版本选择对应的生成方式：

4.3.1 方式一：Android14及之前的版本（ATX机制）

- 修改产品ID

```
cd external/avb/test
```

```
-echo 00000000000000000000000000000000 | xxd -r -p - atx_product_id.bin
+echo 00000000000000000000000000000123 | xxd -r -p - atx_product_id.bin
```

```
cd -
```

注意：产品ID的位数为16字节，数值可以自己定义

- 生成atx_permanent_attributes.bin

```
cd external/avb/test/data
../avb_atx_generate_test_data
cd -
```

执行以上操作后在external/avb/test/data生成：

- atx_permanent_attributes.bin
- atx_metadata.bin
- testkey_atx_pik.pem
- testkey_atx_prk.pem
- testkey_atx_psk.pem

注意：

- pem文件系统中默认有一个，如果需要重新生成，需要删除系统默认的文件，然后再执行上面的操作重新生成pem文件，建议客户自己重新生成
- 这个步骤一个产品只要执行一次就可以，请妥善保管上面生成的文件，在下面的步骤中会使用

4.3.2 方式二：Android15及之后版本（Cert机制）

- 修改产品ID

```
cd external/avb/test
```

```
-echo 00000000000000000000000000000000 | xxd -r -p - cert_product_id.bin
+echo 00000000000000000000000000000123 | xxd -r -p - cert_product_id.bin
```

```
cd -
```

注意：产品ID的位数为16字节，数值可以自己定义

- 生成cert_permanent_attributes.bin

```
cd external/avb/test/data
../avb_cert_generate_test_data
cd -
```

执行以上操作后在external/avb/test/data生成：

- cert_permanent_attributes.bin
- cert_metadata.bin
- testkey_cert_pik.pem
- testkey_cert_prk.pem
- testkey_cert_psk.pem
- testkey_cert_puk.pem

注意：

- pem文件系统中默认有一个，如果需要重新生成，需要删除系统默认的文件，然后再执行上面的操作重新生成pem文件，建议客户自己重新生成
- 这个步骤一个产品只要执行一次就可以，请妥善保管上面生成的文件，在下面的步骤中会使用

4.4 代码修改

4.4.1 device/rockchip BoardConfig.mk修改

进入对应芯片平台的device目录，以RK3576为例：

```
cd device/rockchip/rk3576
```

```
 BOARD_AVB_ENABLE := true     //打开AVB功能
 BOARD_AVB_ALGORITHM := SHA256_RSA4096  //配置签名算法
+# Android14及之前版本(ATX机制)使用以下配置:
+#BOARD_AVB_KEY_PATH := external/avb/test/data/testkey_atx_psk.pem
+#BOARD_AVB_METADATA_BIN_PATH := external/avb/test/data/atx_metadata.bin
+# Android15及之后版本(Cert机制)使用以下配置:
+BOARD_AVB_KEY_PATH := external/avb/test/data/testkey_cert_psk.pem
+BOARD_AVB_METADATA_BIN_PATH := external/avb/test/data/cert_metadata.bin
 #BOARD_AVB_ROLLBACK_INDEX := 5    //配置防版本回滚，默认不开，根据需求开关，需要配合uboot修改
```

芯片平台对应**device**目录参考：

芯片平台	**device**目录
RK3588	device/rockchip/rk3588
RK3576	device/rockchip/rk3576
RK3572	device/rockchip/rk3572
RK3568/RK3566	device/rockchip/rk356x
RK3562	device/rockchip/rk3562
RK3326	device/rockchip/rk3326
PX30	device/rockchip/px30

4.4.2 u-boot defconfig修改

```
cd u-boot
```

```
+# Android14及之前版本(ATX机制)使用:
+CONFIG_AVB_LIBAVB_ATX=y
+# Android15及之后版本(Cert机制)使用:
+CONFIG_AVB_LIBAVB_CERT=y
 CONFIG_AVB_LIBAVB_USER=y
 CONFIG_RK_AVB_LIBAVB_USER=y
+CONFIG_RK_AVB_LIBAVB_ENABLE_ATH_UNLOCK=y
+CONFIG_AVB_VBMETA_PUBLIC_KEY_VALIDATE=y
+CONFIG_ANDROID_AVB_ROLLBACK_INDEX=y //防回滚功能，需要该功能的才配置
```

注意： CONFIG_AVB_LIBAVB_ATX和CONFIG_AVB_LIBAVB_CERT根据Android版本选择其中一个配置，两者不能同时启用。

```
cd -
```

4.5 AVB key烧写

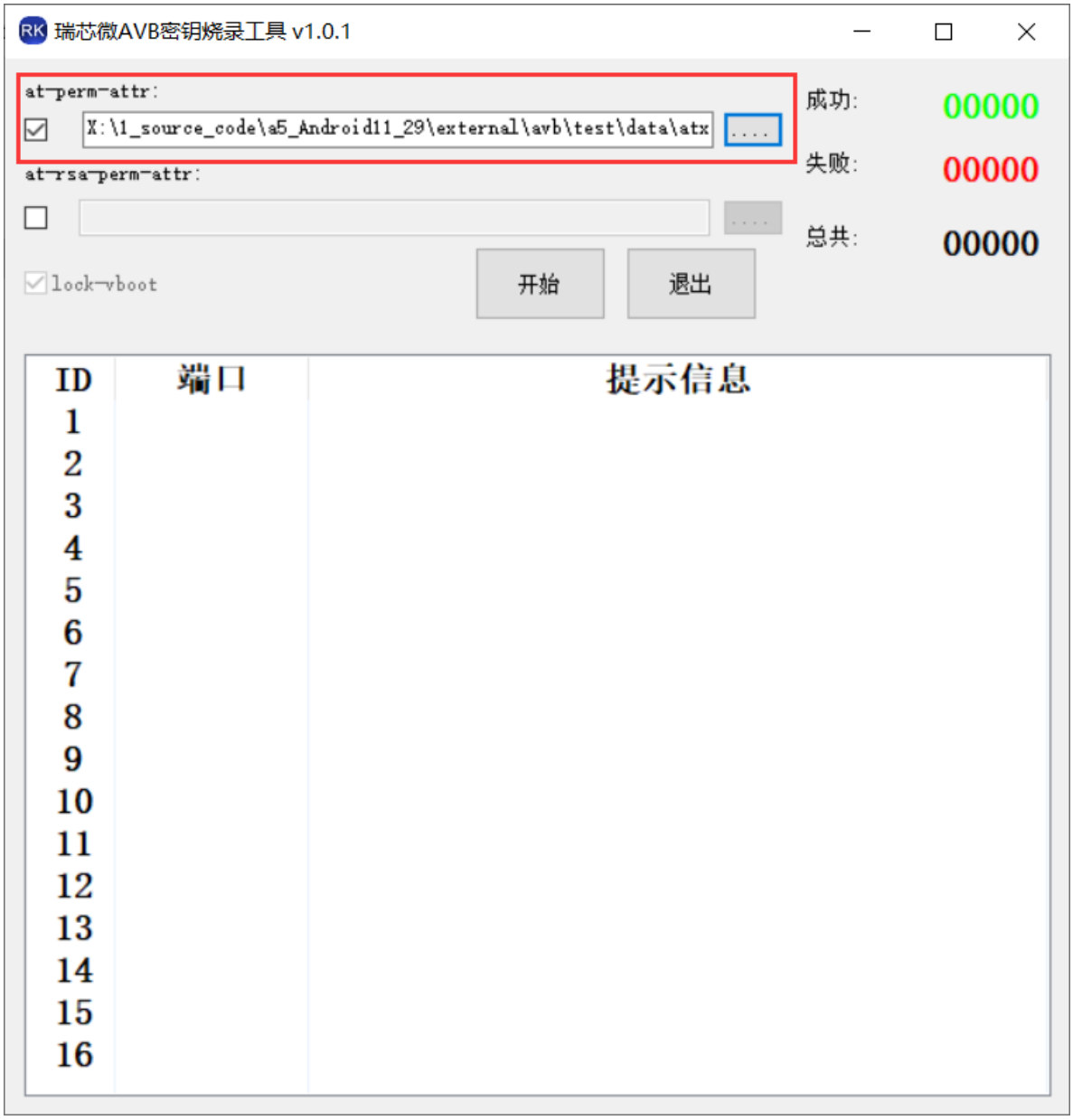
4.5.1 方式一：AVB key烧写工具烧写（推荐量产使用）

根据Android版本选择对应的烧写工具：

4.5.1.1 Android14及之前版本（ATX机制）

烧写工具：AvbKeyWriter（RKTools/windows/AvbKeyWriter-v1.0.1.7z）
烧写源文件：生成的atx_permanent_attributes.bin
烧写方式：

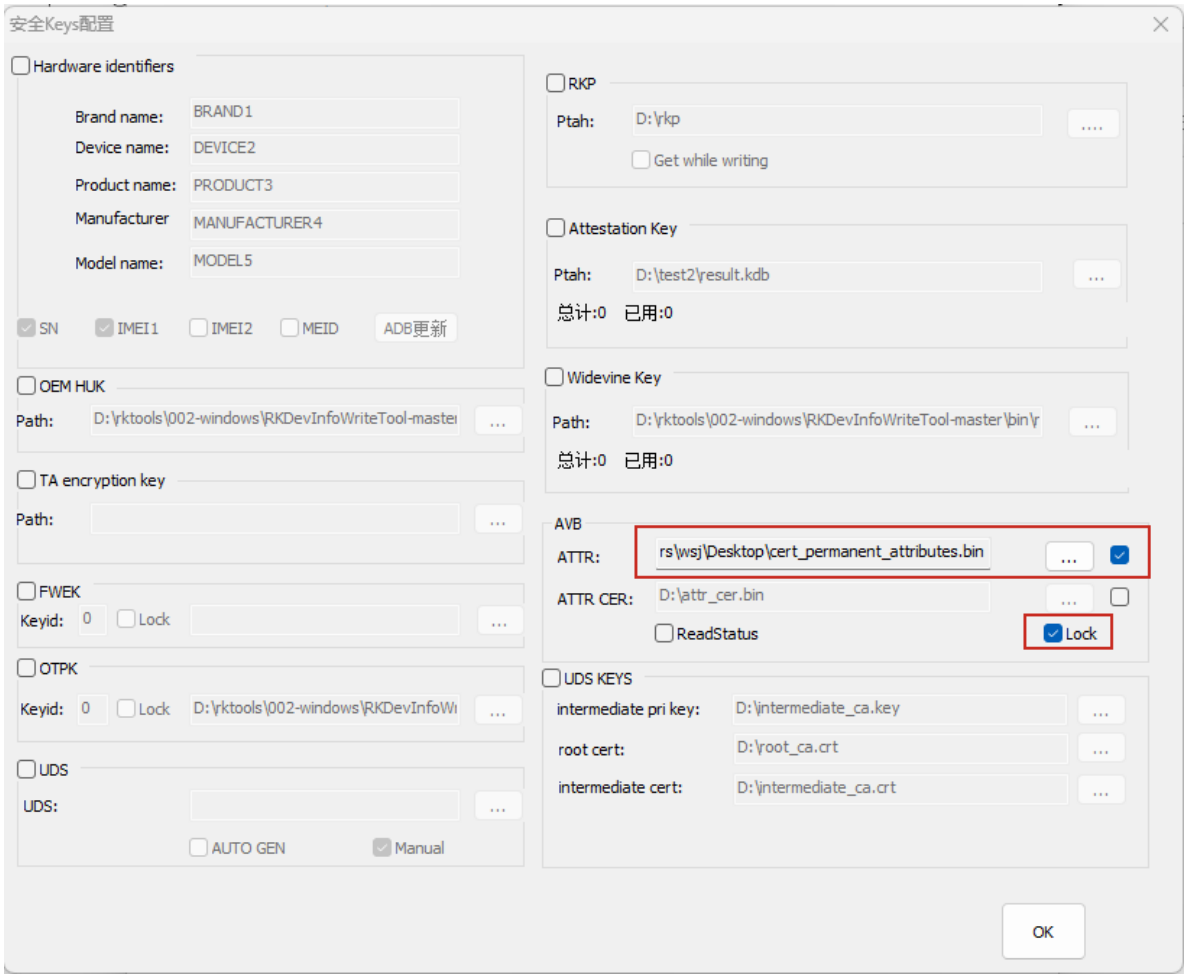
- 勾选at-perm-attr
- 导入atx_permanent_attributes.bin
- 待烧写设备进入loader模式
- 点击"开机按键进行烧写"

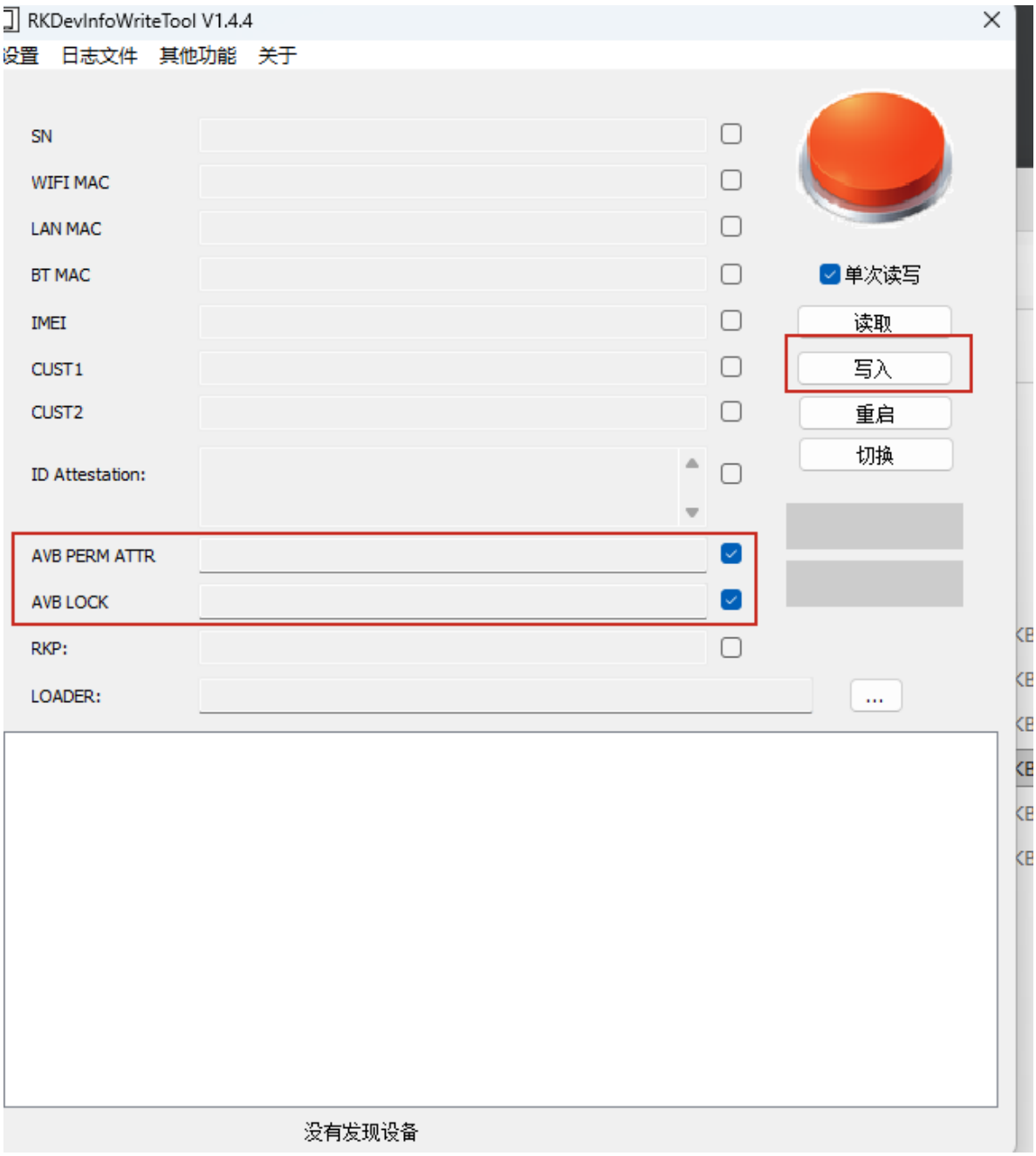


4.5.1.2 Android15及之后（Cert机制）

烧写工具：RKDevInfoWriteTool（RKTools/windows/RKDevInfoWriteTool-v1.4.4_20260522以上版本）
烧写源文件：生成的cert_permanent_attributes.bin
烧写方式：

- 工具→设置→安全keys配置，勾选AVB ATTR
- 导入cert_permanent_attributes.bin
- 待烧写设备进入loader模式
- 点击"开机按键进行烧写"


安全Keys配置
Hardware identifiers
Brand name:
BRAND1
Device name:
DEVICE2
Product name:
PRODUCT3
Manufacturer
MANUFACTURER4
Model name:
MODEL5
SN
IMEI1
IMEI2
MEID
ADB更新
OEM HUK
Path:
D:\rktools\002-windows\RKDevInfoWriteTool-maste
TA encryption key
Path:
FWEK
Keyid:
0
Lock
OTPK
Keyid:
0
Lock
D:\rktools\002-windows\RKDevInfoWi
UDS
UDS:
AUTO GEN
Manual
RKP
Ptah:
D:\rkp
Get while writing
Attestation Key
Ptah:
D:\test2\result.kdb
总计:0 已用:0
Widevine Key
Path:
D:\rktools\002-windows\RKDevInfoWriteTool-master\bin\r
总计:0 已用:0
AVB
ATTR:
rs\wsj\Desktop\cert_permanent_attributes.bin
ATTR CER:
D:\attr_cer.bin
ReadStatus
Lock
UDS KEYS
intermediate pri key:
D:\intermediate_ca.key
root cert:
D:\root_ca.crt
intermediate cert:
D:\intermediate_ca.crt
OK



4.5.2 方式二：AVB key集成到uboot代码内，跟固件一起烧写到机器内

这种方式是将key内置在uboot.img的固件里面，没有烧到安全分区中，当boot.img被替换后key就会失效，所以需要配合SecurityBoot来保证uboot分区的固件不被篡改或者替换。
注意：这种方式不支持严格AVB校验的解锁功能，所以需要解锁请使用方式一。

4.5.2.1 该方式SDK默认没有支持，需要在uboot下打上补丁

根据uboot分支选择对应的补丁：

芯片平台	uboot分支	补丁文件
RK3572	V5	0001-avb-add-embedded-key-for-v5.patch
其他芯片	next-dev	0001-avb-add-embedded-key.patch

补丁路径：RKDocs/common/security/patch/u-boot/

```
cd u-boot
# RK3572使用：
# git am RKDocs/common/security/patch/u-boot/0001-avb-add-embedded-key-for-v5.patch
# 其他芯片使用：
git am RKDocs/common/security/patch/u-boot/0001-avb-add-embedded-key.patch
cd -
```

4.5.2.2 抽取公钥

根据使用的Android版本选择对应的密钥文件：

Android14及之前版本 (ATX)：

```
avbtool extract_public_key --key external/avb/test/data/testkey_atx_psk.pem --output avb_root_pub.bin
xxd -i avb_root_pub.bin > external/avb/test/data/avb_root_pub.h
```

Android15及之后版本 (Cert)：

```
avbtool extract_public_key --key external/avb/test/data/testkey_cert_psk.pem --output avb_root_pub.bin
xxd -i avb_root_pub.bin > external/avb/test/data/avb_root_pub.h
```

4.5.2.3 替换公钥

抽取的公钥（external/avb/test/data/avb_root_pub.h）替换 u-boot/lib/avb/libavb_user/avb_ops_user.c中的avb_root_pub 数组

```
cd u-boot
vim lib/avb/libavb_user/avb_ops_user.c
```

```
/*
* Internal builds use testkey_rsa4096.pem
* OEM should replace this Array with public key used to sign vbmeta.img
*/
static const char avb_root_pub[] = {
0x00, 0x00, 0x10, 0x00, 0x55, 0xd9, 0x04, 0xad, 0xd8, 0x04, 0xaf, 0xe3,
0xd3, 0x84, 0x6c, 0x7e, 0x0d, 0x89, 0x3d, 0xc2, 0x8c, 0xd3, 0x12, 0x55,
...
};
```

```
cd -
```

4.6 固件编译

以上步骤执行完后可以进行完整的固件编译，以RK3576为例：

- 带安全启动的方案

带安全启动的方案uboot要单独先编译，参考上面安全启动的操作步骤

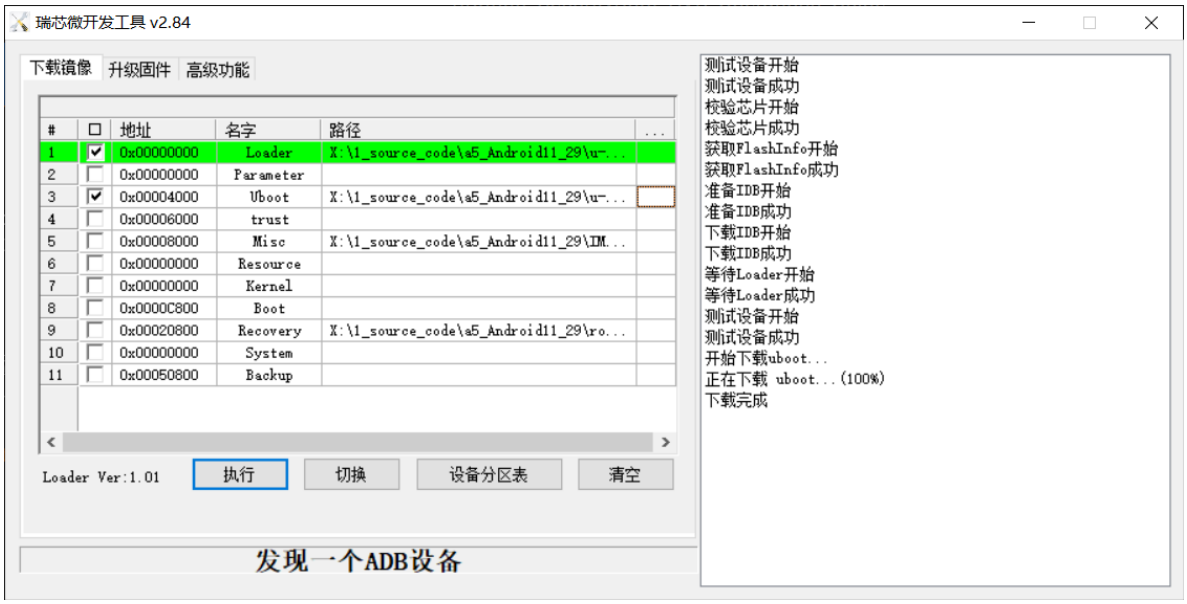
```
source build/envsetup.sh
lunch rk3576_u-userdebug
./build.sh -ACKup
```

- 不带安全启动的方案

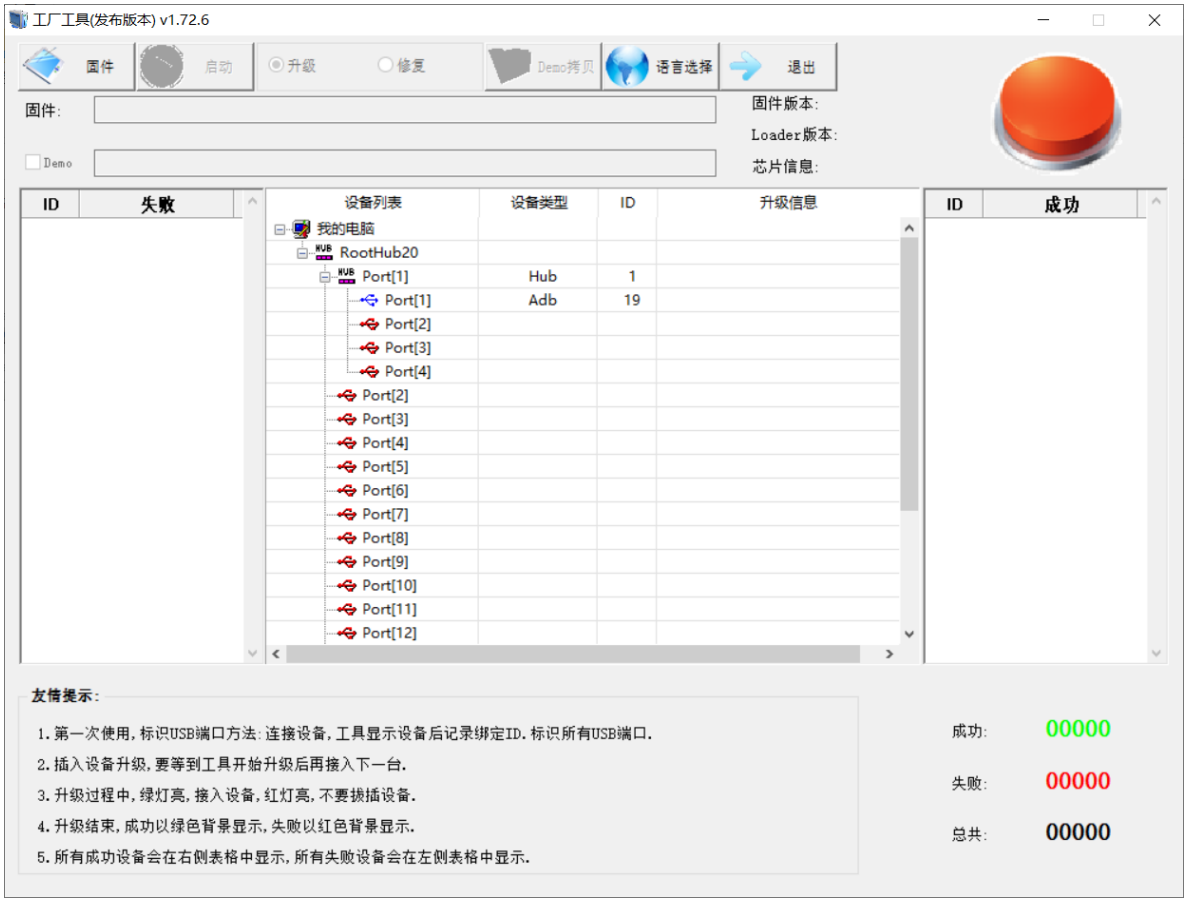
```
source build/envsetup.sh
lunch rk3576_u-userdebug
./build.sh -ACKUup
```

4.7 固件烧写

开发阶段用AndroidTool工具烧写编译出来的完整固件



量产阶段使用量产工具烧写上一步编译生成的update.img



4.8 启动验证

- **uboot开机log确认**

通过以上步骤后系统开机时在串口的log中u-boot阶段会有如下打印：

Android14及之前版本（ATX，不带SecurityBoot）：

```
Vboot=0, AVB images, AVB verify
read_is_device_unlocked() ops returned that device is LOCKED
ANDROID: Hash OK
```

Android15及之后版本（Cert，带SecurityBoot）：

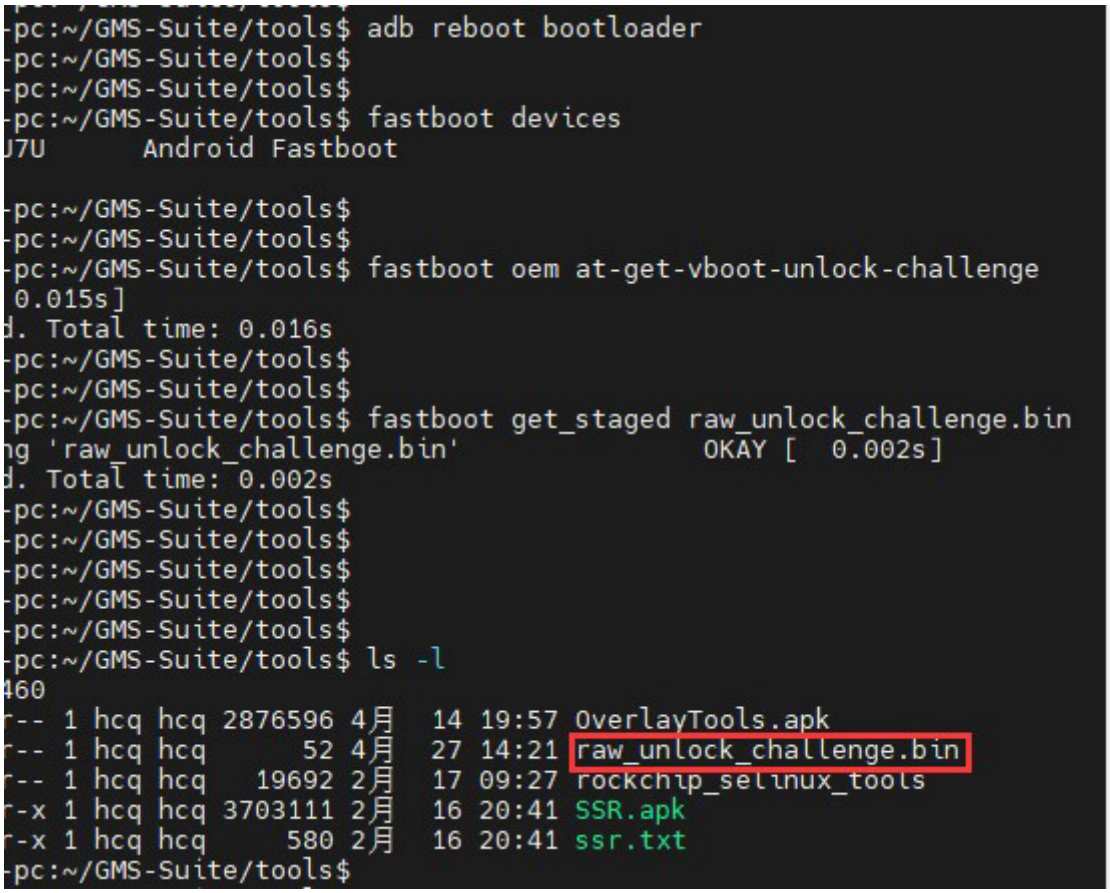
```
Vboot=1, SecureBoot enabled, AVB verify
read_is_device_unlocked() ops returned that device is LOCKED
preloaded(s): pre-full image from 'recovery' at 0xf51f85c0 - 0xf9ea65c0
ANDROID: sha1(a1ea6f6f4f...) + OK
```

- 烧写非AVB固件或者其它的固件会无法开机

4.9 AVB严格校验的解锁步骤

- 设备进入 fastboot 模式，电脑端输入

```
adb reboot bootloader
fastboot oem at-get-vboot-unlock-challenge
fastboot get_staged raw_unlock_challenge.bin
```



获得带版本、Product Id 与 16 字节的随机数的数据，取出随机数生成 raw_unlock_challenge.bin

- 使用 avbtool 生成 unlock_credential.bin

根据使用的Android版本选择对应的product_id文件：

Android14及之前版本(ATX)：

```
python avb-challenge-verify.py raw_unlock_challenge.bin atx_product_id.bin
```

Android15及之后版本 (Cert)：

```
python avb-challenge-verify.py raw_unlock_challenge.bin cert_product_id.bin
```

```
python avbtool make_unlock_credential --output=unlock_credential.bin --
intermediate_key_certificate=external/avb/test/data/testkey_cert_pik.pem --
unlock_key_certificate=external/avb/test/data/testkey_cert_puk.pem --
challenge=unlock_challenge.bin --
unlock_key=external/avb/test/data/testkey_cert_psk.pem
```

其中avb-challenge-verify.py的实现代码如下，请复制代码另存为avb-challenge-verify.py

```
#!/usr/bin/env python
\"\"\"This is a test module for getting unlock_challenge.bin\"\"\"
import sys
from hashlib import sha256
```

```
def challenge_verify():
    if len(sys.argv) != 3:
        print("Usage: avb-challenge-verify.py [challenge_file]
[product_id_file]")
        return
    if sys.argv[1] in ("-h", "--h"):
        print("Usage: avb-challenge-verify.py [challenge_file]
[product_id_file]")
        return
    try:
        with open(sys.argv[1], "rb") as f:
            challenge_data = f.read(52)
        with open(sys.argv[2], "rb") as f:
            product_id_data = f.read(16)
        product_id_hash = sha256(product_id_data).digest()
        print("The challenge version is %d" % challenge_data[0])
        if product_id_hash != challenge_data[4:36]:
            print("Product id verify error!")
            return
        with open("unlock_challenge.bin", "wb") as f:
            f.write(challenge_data[36:52])
        print("Success!")
    except Exception as e:
        print("Error: %s" % str(e))

if __name__ == "__main__":
    challenge_verify()
```

- 烧写unlock_credential.bin，电脑端输入

```
fastboot stage unlock_credential.bin
fastboot oem at-unlock-vboot
```

注意： 此时设备状态一直处于第一次进入 fastboot 模式状态，在此期间不能断电、关机、重启。因为步骤1做完后，设备存储着生成的随机数，如果断电、关机、重启，会导致随机数丢失，后续校验 challenge signature 会因为随机数不匹配失败。

如果开启：

```
CONFIG_MISC=y
CONFIG_ROCKCHIP_EFUSE=y
CONFIG_ROCKCHIP_OTP=y
```

就会使用 CPUID 作为 challenge number，而 CPUID 是与机器匹配的，数据不会因为关机而丢失，生成的unlock_credential.bin 可以重复使用。省去重复生成unlock_challenge.bin，制作unlock_credential.bin的步骤。再次解锁步骤变为：

```
fastboot oem at-get-vboot-unlock-challenge
fastboot stage unlock_credential.bin
fastboot oem at-unlock-vboot
```